

Java Backend Architecture

Interview Series

Chapter 15.6

Saga Pattern

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 15.6 – Saga Pattern

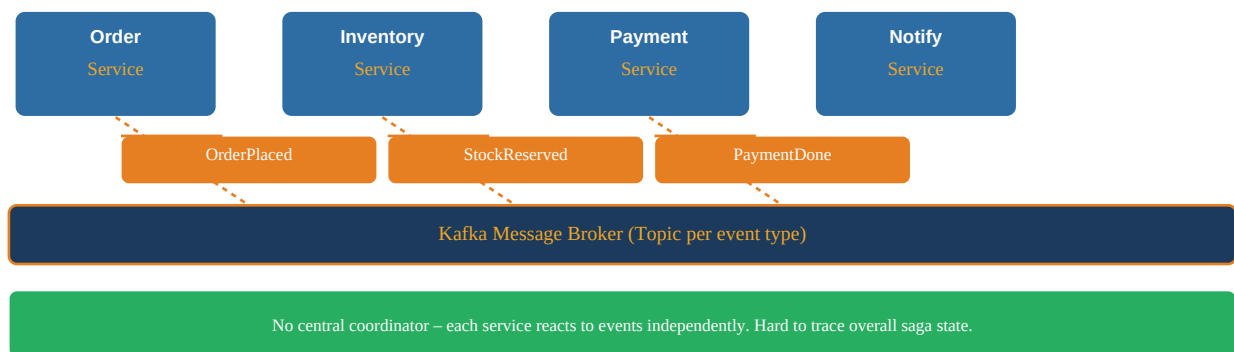
Introduction

The Saga pattern manages distributed transactions across multiple microservices without two-phase commit (2PC). A saga is a sequence of local transactions, each updating one service and publishing an event or sending a command. If any step fails, compensating transactions undo the completed steps. Sagas can be implemented as choreography (event-driven, decentralized) or orchestration (central coordinator). Both maintain data consistency through eventual consistency rather than distributed ACID transactions.

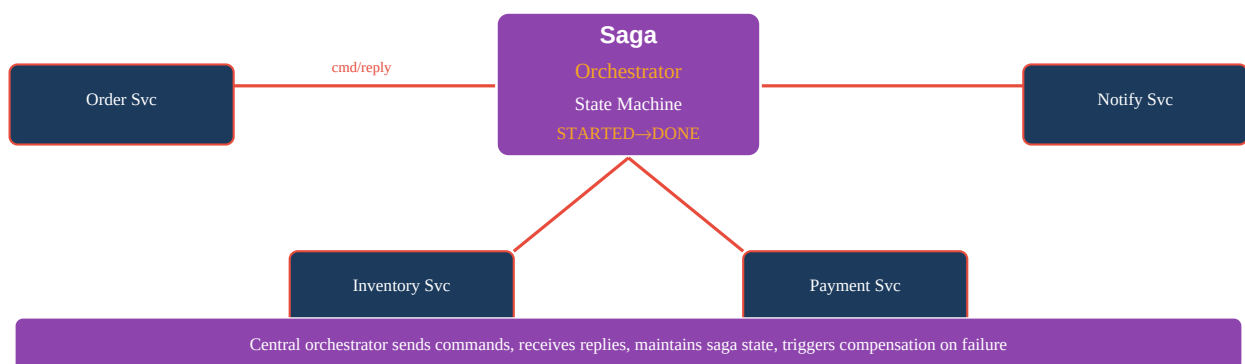
Key Definitions

Saga	A sequence of local transactions across multiple services, coordinated to maintain consistency without distributed ACID transactions. Each step publishes an event or sends a command to the next.
Choreography Saga	Services react to events published by the previous step. No central coordinator. Loose coupling. Hard to trace overall saga state.
Orchestration Saga	Central Saga Orchestrator sends commands to services, receives replies, maintains saga state machine. Easy to trace. Single coordination point.
Compensating Transaction	A new transaction that semantically undoes a completed local transaction. NOT a DB rollback. Example: if payment fails after stock reserved, publish ReleaseStockCommand.
2PC (Two-Phase Commit)	Distributed transaction protocol where a coordinator locks resources across services for an atomic commit. Creates tight coupling, scalability bottleneck, blocking locks. Avoid in microservices.
Saga Orchestrator	Stateful component managing a single saga instance. Sends commands, receives reply events, transitions through state machine (STARTED → STEP1_DONE → COMPLETED/FAILED), triggers compensation.
Eventuate Tram	Java framework (by Chris Richardson) providing transactional messaging, saga orchestration, and orchestration primitives with built-in outbox pattern.
Axon Framework	Java framework supporting Event Sourcing, CQRS, and orchestration sagas. SagaManager, @SagaEventHandler, @StartSaga, @EndSaga annotations.
Saga State Machine	Explicit tracking of saga progress: STARTED, STOCK_RESERVED, PAYMENT_CHARGED, COMPLETED, or FAILED. Persisted to DB by orchestrator. Enables monitoring and recovery.
Pivot Transaction	The step in a saga after which compensation is not possible (or very costly). E.g., once payment is charged, compensation = refund (costly). Pivots define the point of no return.

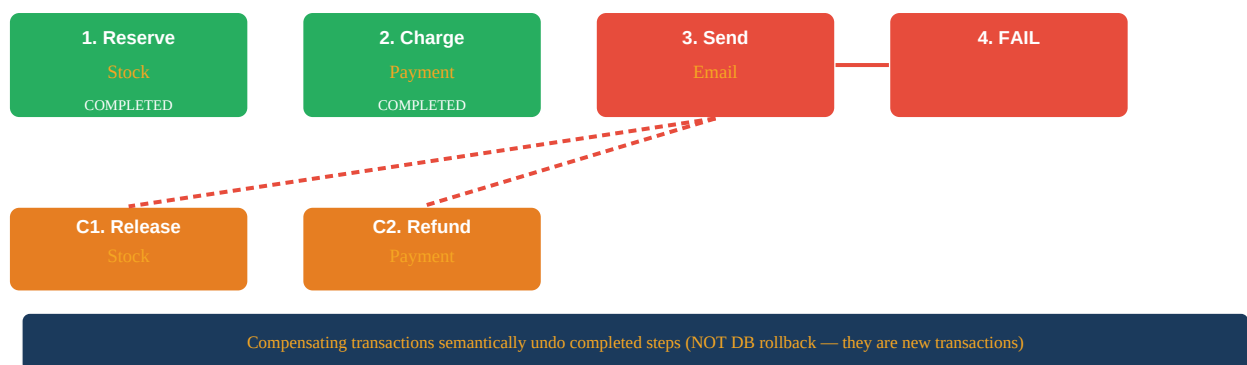
Choreography Saga – Event-Driven Flow



Orchestration Saga – Central Coordinator



Compensating Transaction – Rollback Sequence



Choreography vs Orchestration Trade-offs

Aspect	Choreography	Orchestration
Coordinator	None – services react to events	Central Saga Orchestrator
Coupling	Loose – services know only events	Tighter – orchestrator knows all steps
Traceability	Hard – distributed across services	Easy – orchestrator has full saga state

Failure handling	Compensating events published	Orchestrator directs compensation
Complexity	Simple for short sagas	Better for complex multi-step sagas
Single point of failure	No	Orchestrator (but replicated)
Best for	2-3 step sagas, simple flows	4+ steps, complex compensation
Implementation	Spring Kafka event listeners	Eventuate Tram / Axon Framework

Choreography Saga – Spring Boot

```
// Order Service - starts saga by publishing event @Service @Transactional public class
OrderService { public void placeOrder(PlaceOrderCommand cmd) { Order order = Order.create(cmd);
orderRepository.save(order); // Outbox pattern - triggers saga start outbox.publish(new
OrderPlacedEvent(order.getId(), order.getItems())); } } // Inventory Service - reacts to
OrderPlaced @KafkaListener(topics = "order.placed") @Transactional public void
onOrderPlaced(OrderPlacedEvent event) { try { inventoryService.reserve(event.orderId(),
event.items()); // Success: next step eventBus.publish(new
StockReservedEvent(event.orderId())); } catch (InsufficientStockException e) { //
Compensation: undo order eventBus.publish(new OrderCancelledEvent(event.orderId(),
"InsufficientStock")); } } // Payment Service - reacts to StockReserved @KafkaListener(topics =
"stock.reserved") @Transactional public void onStockReserved(StockReservedEvent event) {
paymentService.charge(event.orderId()); eventBus.publish(new
PaymentProcessedEvent(event.orderId())); } // Order Service - listens for cancellation
(compensation) @KafkaListener(topics = "order.cancelled") @Transactional public void
onOrderCancelled(OrderCancelledEvent event) { orderService.updateStatus(event.orderId(),
OrderStatus.CANCELLED); }
```

Orchestration Saga – Eventuate Tram Style

```
// Saga orchestrator definition @Saga public class PlaceOrderSaga { private static
SagaDefinition<PlaceOrderSagaData> sagaDef = step() .invokeParticipant(data ->
reserveStockCommand(data)) .onReply(StockReservedReply.class, (d, r) ->
d.setStockReservationId(r.getId())) .withCompensation(data -> releaseStockCommand(data))
.step() .invokeParticipant(data -> chargePaymentCommand(data))
.onReply(PaymentChargedReply.class, (d, r) -> d.setPaymentId(r.getPaymentId()))
.withCompensation(data -> refundPaymentCommand(data)) .step() .invokeParticipant(data ->
confirmOrderCommand(data)) .build(); @Override public SagaDefinition<PlaceOrderSagaData>
getSagaDefinition() { return sagaDef; } } // Start saga from order service @Service
@Transactional public class OrderService { @Autowired SagaManager<PlaceOrderSagaData>
sagaManager; public String placeOrder(PlaceOrderRequest req) { Order order =
orderRepository.save(Order.create(req)); PlaceOrderSagaData sagaData = new
PlaceOrderSagaData(order.getId(), ...); sagaManager.create(sagaData); // Orchestrator persists
state + sends first command return order.getId(); } }
```

Interview Q&A;

Q1. What is the Saga pattern and why is it needed?

A Saga is a sequence of local transactions across multiple microservices that maintains data consistency without distributed ACID transactions (2PC). Each local transaction updates one service and triggers the next. Needed because: 2PC requires locking resources across services (scalability bottleneck, tight coupling, single point of failure). Sagas use eventual consistency with compensating transactions instead.

Q2. What is the fundamental difference between choreography and orchestration sagas?

Choreography: each service publishes events; next service reacts independently. No central coordinator. Decoupled, but hard to trace the overall saga state. Orchestration: central Saga Orchestrator sends explicit commands to services, receives replies, maintains the saga state machine. Easy to trace. Single point of coordination.

Q3. Why is 2PC (Two-Phase Commit) avoided in microservices?

2PC requires a coordinator to lock resources across multiple services simultaneously. Problems: (1) Locks held during network calls — scalability bottleneck, (2) Coordinator failure leaves resources locked, (3) Services must implement XA protocol — tight coupling, (4) Long-held locks reduce throughput. Sagas replace 2PC with eventually consistent local transactions + compensating transactions.

Q4. What are compensating transactions?

New transactions that semantically undo the effect of previously completed local transactions. NOT database rollbacks — the original transaction already committed. Example: if payment fails after stock was reserved, a compensating transaction releases the reserved stock. Compensating transactions must be idempotent (safe to execute multiple times).

Q5. What is a saga state machine?

The orchestrator maintains explicit state for each saga instance: `STARTED` → `STOCK_RESERVED` → `PAYMENT_CHARGED` → `ORDER_CONFIRMED` → `COMPLETED`. Or failure states: `STOCK_RESERVATION_FAILED`, `PAYMENT_FAILED`, `COMPENSATING`, `COMPENSATION_COMPLETE`. State persisted to DB. Enables monitoring, debugging, and recovery after orchestrator restart.

Q6. How do you handle idempotency in sagas?

All saga steps and compensating transactions must be idempotent — safe to execute multiple times. At-least-once delivery means the same command may arrive twice. Implementation: include a unique saga ID + step ID with each command. Services check if this saga step was already executed (deduplication table) before processing.

Q7. What is the difference between forward recovery and backward recovery in sagas?

Forward recovery (retry): if a step fails due to transient error (network timeout), retry with backoff. Backward recovery (compensate): if a step fails due to business rule violation (insufficient stock), trigger compensating transactions to undo previous steps and reach a consistent `FAILED` state. Orchestrator decides which recovery to use based on error type.

Q8. How does the Eventuate Tram framework simplify saga implementation?

Eventuate Tram provides: (1) Transactional messaging via outbox pattern built-in, (2) SagaManager with persistent saga state, (3) Declarative saga definition (`step().invokeParticipant().withCompensation()`), (4) Command routing to correct service, (5) Reply handling and state transitions. Eliminates boilerplate for outbox, state persistence, and compensation tracking.

Q9. What is the pivot transaction in a saga?

The point of no return — after this step, rolling back becomes very costly or impossible. Example: after payment is charged, compensation = refund (triggers bank reversal process, customer email). Design principle: arrange saga steps to minimize the cost of compensation. Put high-risk steps early so compensating is cheaper.

Q10. How do you trace a saga execution across multiple services?

Include the saga ID in all commands, events, and log statements. Each service logs with the saga ID. Distributed tracing propagates the saga ID as a correlation ID through Kafka message headers. Kibana/Grafana Loki: filter by saga ID to see full execution trace. Orchestrator-style sagas: centralized state

shows all steps and their status.

Q11. What happens when a saga orchestrator crashes mid-saga?

Orchestrator persists saga state to DB before each step. On restart: reads all in-progress sagas, determines last completed step, resumes from there. Idempotent steps: safe to retry the last step even if it may have already executed. This is why deduplication (idempotency keys) in participant services is critical.

Q12. Describe a saga for an e-commerce order placement.

Steps: (1) Create Order (DRAFT status) — OrderCreated event. (2) Reserve Stock — InventoryService, reply StockReserved or InsufficientStock. (3) Charge Payment — PaymentService, reply PaymentCharged or PaymentFailed. (4) Confirm Order (CONFIRMED status). (5) Send Confirmation Email — NotifyService. Compensations: for step 3 failure → release stock. For step 4 failure → release stock + cancel order.

Q13. What is an 'isolation anomaly' in sagas?

Unlike ACID transactions, sagas don't have isolation. While a saga is executing, other operations can see intermediate state (order in DRAFT status, stock reserved but not paid). Mitigation: (1) Semantic locks (mark data as PROCESSING), (2) Commutative updates (design operations that can run in any order), (3) Pessimistic view (assume worst case), (4) Version-based reread (optimistic lock check before final commit).

Q14. How does Axon Framework support sagas?

Axon: @Saga annotates the saga class. @StartSaga on the method that starts the saga. @SagaEventHandler for event handling within the saga. @EndSaga when saga completes. @Autowired on participant command gateways within the saga. Axon Server handles event routing and saga state persistence. SagaLifecycle.associateWith() links external IDs to the saga.

Q15. What is the semantic lock countermeasure for saga isolation?

Acquire a semantic lock (flag in DB) at the start of a saga step. Other operations that would conflict must wait for the lock to be released. Example: Order status = PROCESSING prevents double-purchase. Lock released when saga step completes or compensating transaction runs. Not as strong as ACID isolation but prevents obvious concurrency issues.

Q16. When should you use choreography vs orchestration?

Choreography: 2-3 step sagas, simple flows, team prefers event-driven decoupling, services are genuinely independent and don't need central coordination. Orchestration: 4+ steps, complex compensation logic, need clear visibility into saga state, regulatory/audit requirements need explicit saga tracking, cross-team sagas where one team owns the orchestrator.

Q17. How do you handle timeouts in sagas?

Each saga step has a timeout. If no reply received within timeout: (1) Retry (transient failure) — exponential backoff. (2) Fail (trigger compensation) — treat as permanent failure. Orchestrator uses a timer/scheduler to detect timeout. Spring: ShedLock + @Scheduled to periodically check for timed-out sagas. Eventuate Tram: built-in timeout handling.

Q18. What is the difference between a saga and a workflow?

Saga: focused on distributed transactions with compensation for failure. Business-transaction semantics. Workflow: broader concept — includes human tasks, approvals, long-running processes, branching, parallel execution, time-based triggers. Tools: Camunda, Temporal. Sagas can be implemented as workflows in Temporal/Camunda for complex orchestration needs.

Q19. What is countermeasure for 'dirty reads' in sagas?

Dirty read: service B reads data written by saga A before saga A commits all steps. Countermeasure: commutative updates — design updates so order doesn't matter. Or: aggregate-level locking — lock the aggregate until saga completes. Or: PENDING status on resources — show as PENDING until saga commits, not as final state.

Q20. How do you test saga compensation logic?

Unit: inject saga with mock participants, simulate failure at each step, verify correct compensating commands are sent. Integration (Testcontainers + Kafka): run full saga with real services, inject failure at specific step, verify compensation executes and system reaches consistent state. Test each compensating transaction in isolation for idempotency.

Q21. What is a 'lost update' anomaly and how do sagas handle it?

Lost update: two sagas concurrently update the same aggregate, second overwrites first. Mitigation: optimistic locking (version field on aggregate). If saga tries to apply an update and version is wrong, it means another saga ran concurrently. Saga must retry (if possible) or fail with compensation.

Q22. How does Kafka support saga choreography?

Kafka topics per event type (order.placed, stock.reserved, payment.processed). Each service subscribes to topics relevant to its role. Messages include `sagald` as header for correlation. Kafka guarantees ordering within partition (use `sagald` as partition key — all saga events in one partition). Consumer groups ensure each service processes events independently.

Q23. What is a saga log?

Persistent record of all saga steps, their status, inputs, outputs, and timestamps. Maintained by the orchestrator or a dedicated saga log service. Provides audit trail for compliance. Enables saga recovery after failure. Enables customer service (show customer order processing status). Query: `SELECT * FROM saga_log WHERE saga_id = 'order-123' ORDER BY step_number`.

Q24. What is the difference between saga and 2PC in terms of data consistency?

2PC: synchronous, ACID isolation, all-or-nothing commit. Resources locked until all participants agree. Strong consistency but low availability and scalability. Saga: asynchronous, eventual consistency, no locks across services. Each step commits independently. If failure: compensating transactions create a new consistent state. High availability and scalability, but intermediate states are visible.

Q25. How do you handle duplicate commands in saga participants?

Include saga ID + step number in every command. Participant checks deduplication table: `SELECT id FROM processed_saga_steps WHERE saga_id=? AND step=?`. If already processed: return cached reply without reprocessing. If not: process, insert into deduplication table, return new reply. Check + process + insert must be in the same database transaction.

Q26. What is the 'saga rollback' anti-pattern?

Trying to achieve true rollback semantics in a saga — impossible once services have committed. Compensating transactions create a new consistent state, they don't literally undo. Example: compensation for 'send email' is 'send cancel email', not 'un-send the email'. Design sagas to accept eventual consistency, not to emulate synchronous rollback.

Q27. How do you monitor saga health in production?

Track: sagas started, completed, failed (compensation triggered) per time period. Alert: sagas stuck in same state for more than X minutes (likely deadlock/crash). Alert: compensation rate spikes (business problem or system degradation). Dashboard: saga state distribution (how many in STEP1, STEP2, COMPLETED, FAILED). Tools: Prometheus counter per state transition, Grafana dashboard.

Q28. What happens to a saga when a compensating transaction fails?

If compensation fails, the system is in an inconsistent state. Options: (1) Retry compensation with backoff (most common — eventual success). (2) Alert operations team for manual intervention. (3) Schedule compensation retry via persistent retry queue. Critical: log all failed compensations immediately for audit/ops. Design compensating transactions to be as reliable as possible.

Q29. Describe saga pattern in the context of payment processing.

Steps: (1) Validate order (2) Reserve funds (soft hold on payment instrument) (3) Reserve inventory (4) Confirm payment (actual charge) (5) Update order status (6) Notify customer. Compensations: (4) failure → release fund hold + release inventory. (2) failure → cancel order. The pivot transaction is step (4) — actual charge triggers bank's own process. After (4): compensation = issue refund, not cancel charge.

Q30. What is the benefit of using a framework like Temporal for sagas?

Temporal provides: durable execution — saga code runs as if never fails, automatic retries with backoff, activity timeouts, workflow versioning, queryable workflow state, replay-based debugging. Eliminates most saga boilerplate. Java SDK: `@WorkflowInterface`, `@ActivityInterface`, `Workflow.newActivityStub()`. Better for complex long-running sagas than custom Eventuate/Axon implementations.

Q31. How do you version saga definitions?

When saga logic changes (add/remove steps), in-flight sagas may be using old definition. Options: (1) Drain old sagas (wait for completion before deploying new version). (2) Version saga state machine (if `STEP_3_OLD`, use old logic; if `STEP_3_NEW`, use new). (3) Temporal: `Workflow.getVersion()` API explicitly handles workflow versioning. Never change saga definition for in-progress sagas without version handling.

Q32. What are the trade-offs of saga vs 2PC for order management?

2PC: stock and payment locked simultaneously → guaranteed consistency. But: DB lock contention under load, payment gateway timeout = entire order stalls. Saga: stock reserved, payment charged separately → visible intermediate states, compensation needed if payment fails. But: no cross-service locks, payment gateway failure only affects payment step (not stock service). For e-commerce scale: sagas are the only viable option.

Q33. How do you implement a saga in pure Spring Boot without a framework?

Define saga state enum (`STARTED`, `STOCK_RESERVED`, `PAYMENT_CHARGED`, `COMPLETED`, `FAILED`). Persist saga instance to DB with current state. Kafka listeners at each step: on success, update state and publish next command. On failure, publish compensation commands and transition to `COMPENSATING` state. Scheduler checks for stuck sagas (state unchanged for > timeout) and retries. More work than frameworks but gives full control.

Q34. What is the relationship between the outbox pattern and sagas?

The outbox pattern ensures that local DB write and event publishing are atomic. In choreography sagas: each step uses outbox to guarantee event is published after local transaction commits. In orchestration sagas: orchestrator uses outbox to guarantee commands are sent after saga state is persisted. Without outbox: saga may miss a step if service crashes between DB commit and event publish, leading to saga getting stuck.

Q35. What is a saga's interaction with read models (CQRS)?

Each saga step typically updates the write model. Domain events emitted by saga steps drive read model updates via CQRS projections. Example: `OrderStatusUpdatedEvent` → `OrderStatusProjection` updates read model status. Customer can query read model for live saga progress. Saga + CQRS + Event Sourcing is a powerful combination for complex domain workflows.

Q36. Describe compensating transactions for a hotel booking saga.

Steps: (1) Reserve room → Compensation: cancel room reservation. (2) Reserve flight → Compensation: cancel flight reservation. (3) Charge credit card → Compensation: refund credit card. (4) Send booking confirmation → Compensation: send cancellation email. Pivot: step (3) — credit card charge triggers external system. If step (4) fails: send cancellation email (compensation), issue refund (comp step 3). If step (2) fails: cancel room reservation (comp step 1). No payment was made yet.

Q37. What is the 'countermeasure' pattern in saga design?

Strategies to reduce anomalies caused by lack of saga isolation: (1) Semantic locks: mark affected resources as PROCESSING to block conflicting operations. (2) Commutative updates: order operations so any sequence produces same result. (3) Pessimistic view: assume worst case to prevent most dangerous anomalies. (4) Reread value: re-read resource before update to detect concurrent modification. (5) Version files: version records and apply commutative updates.

Q38. What are the testing challenges unique to sagas?

Sagas are distributed, asynchronous, and stateful — hard to test in isolation. Challenges: (1) Reproducing failure at specific saga step, (2) Verifying compensation executes correctly, (3) Testing timeout scenarios, (4) Testing concurrent sagas with isolation anomalies, (5) Saga state recovery after crash. Testcontainers with embedded Kafka, injected failure via mock participants, and deterministic timing help.

Q39. What is the role of the saga ID in distributed systems?

Unique identifier for a saga instance. Included in all events, commands, and logs related to the saga. Used for: (1) Correlation across services, (2) Idempotency checking in participants (process each step once), (3) Distributed tracing (filter all logs/spans for one saga), (4) Saga state lookup in orchestrator DB, (5) Customer-facing order tracking.